Linearna Regresija
Formula: $h_\theta = \theta_0 + \theta_1 x$. Parametri ovog modela su θ_0 i θ_1

Linearna regresija je nadgledani model θ_0 je presek θ_1 je nagib.
Teta 0 nije interpolabilno u opstem slucajju. U matematickom slucaju teta 0 je ocekivana vrednost y kada su x postavljeni na nula, a taj primer uglavnom nije prakticno moguc. Ne treba koristiti regresioni model za predikciju tacaka van opsega trening podataka.(Interpolacija je proces procenjivanja vrednosti unutar opsega poznatih podataka, a ekstrapolacija je proces procenjivanja vrednosti van opsega poznatih podataka). Drugim recima ako nemamo trening podatke u okolini 0 vrednosti ne vredi da interpretiramo teta 0. Cak i ako imamo teta 0 ne mora biti interpolabilno, ali je od krucijalnog znacaja da teta 0 uvrstimo u model, u suprotnom forsiramo regresionu liniju da prodje kroz koordinatni pocetak. **Vrednost regresionog modela je razumevanje kako se y menja sa promenom x**.

Najbolju liniju u linearnoj regresiji odabiramo obucavanjem modela. (Rezidual je razlika izmedju posmatrane vrednosti i vrednosti koju je predvideo model). Obucavanje modela vrsicemo izracunavanjem cene za svaku liniju i onda cemo naci teta za koje je cena najmanja. RSS je residual sum squares.

$$RSS(\theta) = \sum_{i=1}^{n} (y_i - (\theta_0 + \theta_1 x_i))^2$$

Gradijentni spust

Ulaz	• $J(\theta)$ – funkcija koja se optimizuje • θ_0 – početno rešenje • α – *learning rate* (veličina koraka) • *maxIters* – maksimalan broj iteracija
Postupak	for t = 1, 2,..., maxIters: for d = 1,2,...,D $\theta_d^{(t+1)} = \theta_d^{(t)} - \alpha \frac{\partial}{\partial \theta_d} J(\theta)$
Izlaz	θ – tačka u kojoj funkcija $J(\theta)$ ima minimum

- Gradijentni spust pronalazi lokalni optimum, dakle u zavisnosti od inicijalizacije mozemo zavrsiti u razlicitim tackama.

- Da li Gradient Descent u slučaju linearne regresije sa RSS merom greške može da zaglavi u lokalnom minimumu? Pa ne moze zato sto je RSS kvadratna funkcija u parametrima teta 0 i teta 1 je konveksna i lici na parabolu koja ima jedan minimum.

- Kod obucavanja modela, za male vrednosti learning rate gradient descent je spor, a za prevelike vrednosti divergira.
- Stepsize schedule je smanjivanje koraka learning rate sa brojem iteracija. n=learning rate/t ili n= learning rate/sqrt(t).

- Kako bi gradijentni spust brze konvergirao potrebno je normalizovati vrefdnost obelezja

- Magnituda promene je proporcionalna gresci tj ako je mala greska i teta cemo malo menjati, ako je velika mnogo cemo menjati teta.

Mean normalization
Skalirati vrednosti obelezja tako da srednja vrednost bude bliska nuli

$$x_d = \frac{x_d - \min(x_d)}{\max(x_d) - \min(x_d)}$$

$$x_d = \frac{x_d - \mu}{\sigma}$$

- μ - srednja vrednost obeležja x_d
- σ - standardna devijacija obeležja x_d

LMS
- Analiticko pronalazenje resena, Pronalazenje ekstrema(min) -> izjednacavanje pracijalnih izvoda sa 0

Batch Graduient Descent
- Koristimo ceo skup podataka da azuriramo model. Mana je sto je iteracija podataka skupa ako je N veliko

Stohastic Gradient Descent
- Vise puta prolazimo kroz askup podataka, azuriramo model na osnovu jednog treninhg podatka
- Napredujemo sa svakim podatkom, dakle ovo je preferirano ako je N veliko.
- Obicno stohastic ce dovesti teta brze blizu minimuma od batch GD, i zzato ga koristimo kada je N veliko.

Normal Equation VS Gradient Descent
- NE nema parametre, kod GD biramo alfa
- NE jedna iteracija, GD mnogo
- NE $(X^TX)^{-1}$ je sporo za veliko D, GD vreeme ne zavisi od D
- NE X^TX moze biti singularno

Outlieri
- Ekstremne vrednosti mogu i ne moraju da uticu na model. one koje uticu zovemo uticajne tacke.
- Potencijal za high leverage point je ako je x ekstremno veliko/malo i ako nema drugih tacaka u blizini

- Uklonicemo outliere ako smo sigurni da je greska
- ako imamo mnogo podataka a malo outliera
- Postoje i alternative za uklanjanje Winsorizin, log-scale transformation, binning imputing values.
- Treniramo poseban model ako ima mnogo outliera, i umesto manipulacije podacima koristimo robusan model i promenimo meru performansi

Funkcija greske

- Mera koliko je dobar model u predvidjanju ocekivanog ishoda
- Zavisi od outliera, modela, vremenske efikasnosti optimizacionog algoritma, jednostavnosti pronalaska izvoda i od samog problema
- Ako imamo korumprirane podatke koristimo MAE, ako su anomalije bitne i treba ih pokriti koristimo MSE
- **Metrike su MAE MSE R2**

Visestruka regresija

Polinomijalna regresija

i dalje spada pod linearnu regresiju jer je linearna po teta

Model Obeležje

$$h_\theta(x) = \theta_0 f_0(x) + \theta_1 f_1(x) + \theta_2 f_2(x) + \cdots + \theta_D f_D(x) = \sum_{d=0}^{D} \theta_d f_d(x)$$

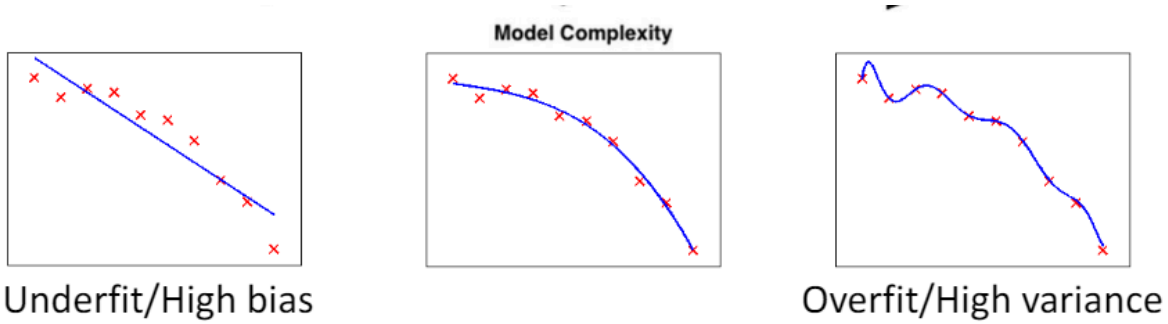
$$h_\theta(x) = \sum_{d=0}^{D} \theta_d x^d$$

(zbog jednostavnosti notacije uz θ_0 ide $x^0 = 1$)

Ka ko znamo do kog stepena da idemo? Pa najbolje znamo uz pomoc sanitation,vaccination, BMI, kriminal.. ali imamo i D-dimenzioni prostor koji ne mozemo vizualizovati. Najbolji nacin da to vizualizujemo je funkcija gubitka

Stratifikacija

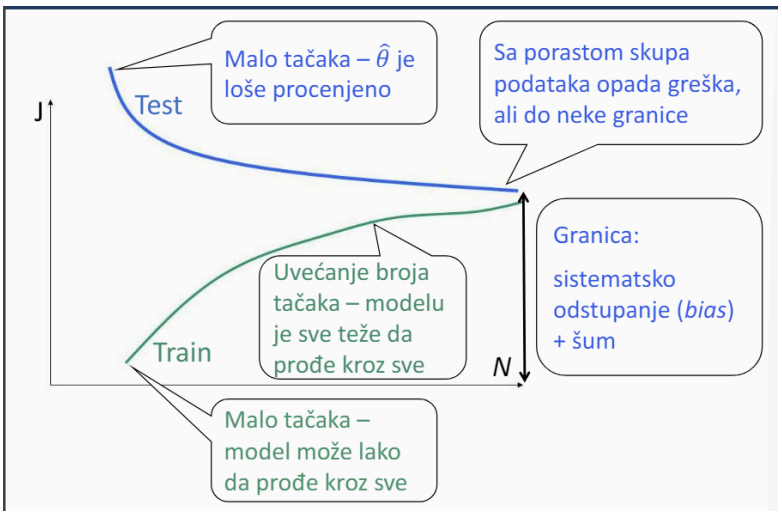
prilikom deljenja podataka obezbediti da delovi imaju istu raspodelu kao i ceo skup podataka, sto je tesko za male podatke



Overfitting

Previse obelezja, a premalo podataka dovodi do toga da se hipoteza jako dobro uklapa u trening podatke ali jako lose generalizuje na nove primere. Za model kazemo da je overfitovan ako postoji model2 takav da je 1. trening greska prvog modela < trening greske drugog modela, ali prava greska prvog modela> prave greske drugog modela

Preprilagodjavanje nije vezano samo za uvodjenje visih stepena polinoma, ono se javlja i kao posledica prevelikog broja obelezja.

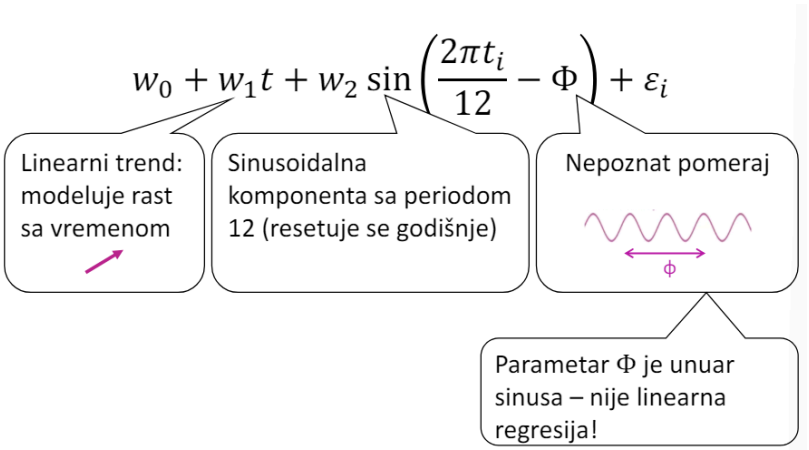


ZAKLJUCAK

- Da bismo izbegli preprilagodjavanje moramo imati reprezentativan skup
- za jednu varijablu ovo je jako tesko izvesti
- problem se povecava povecavanjem broja ulaza
- upravljanje prilagodljivoscu modela je od kljucnog znacaja za dobru generalizaciju

Sezonalnost

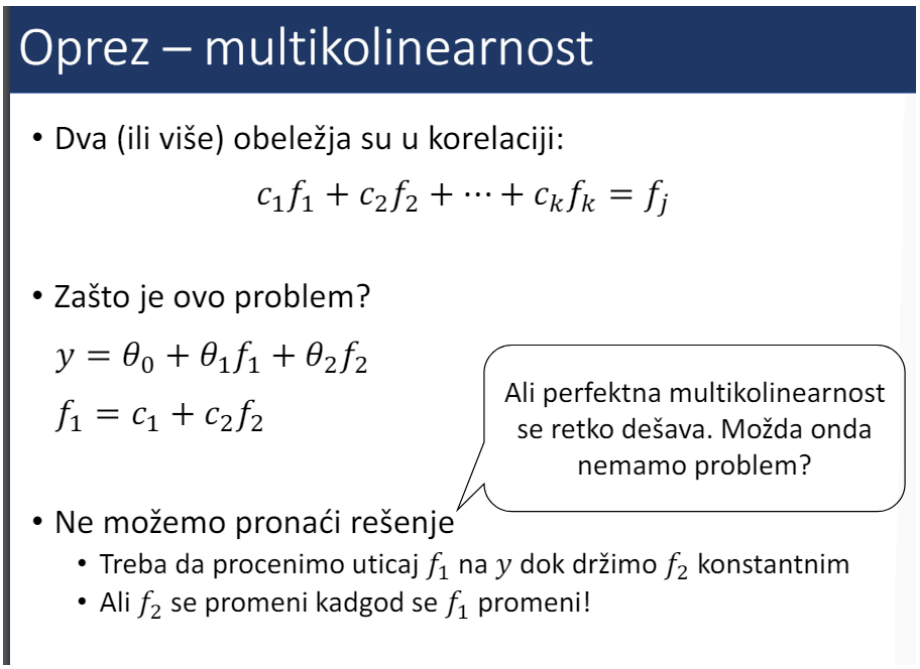
- sablon koji se ponavlja



- Kad imamo jednu ulaznu varijablu mozemo uvecati fleksibilnost dodavanjem visih stepena polinoma, a mozemo dodati i vise nezavisnih varijabli
- Dodavanjem obelezja dobijamo na kompleksnosti modela, sto je dobro jer modeli niske kompleksnosti nisu dovoljno fleksibilni da opisu podatke, ali kompleksni modeli mogu da pade od overfittinga (velika varijansa malo sistemsko odstupanje)
- overfitting se desava kada imamo veliki broj obelezja D a premalo trening podataka N
- znak overfittinga - velika magnitujda koeficijenata modela teta
- **velika varijansa malo sistemsko odstupanje(bias) overfitting**
- **mala varijansa veliko sistemsko odstupanje(bias) underfitting**

Kategoricka obelezja

- ako ne postoji prirodan poredak (nominalna) onda one hot encoding
- ako postoji prirodni poredak (ordinalna) onda label encoding

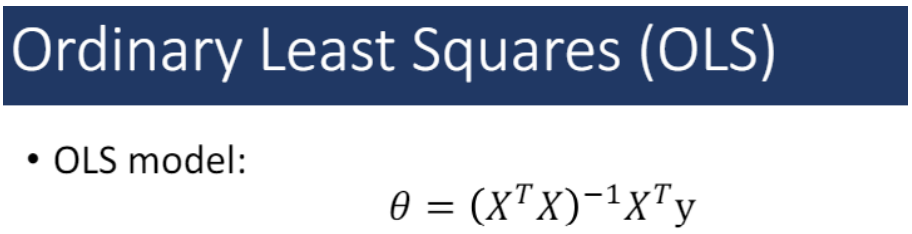


ovo moze biti problem jer teramo model da uci tezinu koja mu ne treba, dolazimo do prokletstva dimenzionalnosti, model je teze interpretirati.

- teta su osetljive na specificnosti trening skupa, ako se uklone varijable promenice se drasticno
- resenje sa one hot encodingo izbaciti jednu od tri variable

03 Regularizacija

Ordinary least squares



- problem moze biti singularna matrica kada je broj obelezja veci od broja instanci ili kada su prediktori multikolinearni
- singularna matrica znaci da je sistem neodredjen i da imamo beskonacno mnogo resenja i ne znamo koje da odaberemo
- multikolinearnost je kada jedno obelezje predstavlja linearnu kombinaciju drugih obelezja
- kada imamo perfektnu multikolinearnost XtX nema resenje, to se retko desava
- **za dobru generalizaciju je kljucno upravljanje prilagodljivoscu modela**
- dobra prilagodjenost trening podacima ne znaci da je dobra generalizacija
- kada imamo **veliku varijansu** male promene trening skupa rezultuju veoma razlicitim modelima
- **preprilagodjavanje resavamo uvecavanjem broja podataka, smanjenjem dimenzionalnosti i regularizacijom**

- ali smanjenjem dimenzionalnosti ne smemo izbaciti obelezja bitna za resenje, takodje kompleksnost modela treba da odgovara kompleksnosti modela a ne kolicini podataka

Regularizacija

- sacuvati sva obelezja ali smanjiti magnitude vrednosti teta
- ovo rezultuje jednostavnijom hipotezom koja je manje podlozna preprilagodjavanju, radi dobro u slucaju kada imamo mnogo obelezja takvih da svako dobrinosi predikciji
  - U slučaju linearnih modela

$$h_\theta(x) = \sum_{d=1}^{D} \theta_d x_d$$

važi:

$$\theta = \nabla_x h_\theta(x)$$

Efekat regularizacije:
ograničavamo gradijent funkcije $h_\theta(x)$ (brzinu promene funkcije $h_\theta(x)$ sa promenom x)

- regularizacija je bilo koja modifikacija optimizacionog problema koja ogranicava prilagodljivost modela i cini ga manje podloznim overfittingu. U jos opstijem smislu regularizacija je bilo kakva modifikacija matematickog problema koja ga cini bolje uslovljenim
  - Modifikacija funkcije greške: umesto direktne minimizacije gubitka

$$L\left(h_\theta\left(x^{(i)}\right), y^{(i)}\right)$$

minimizujemo regularizovanu grešku:

$$\min_\theta \frac{1}{N} \sum_{i=1}^{N} L\left(h_\theta\left(x^{(i)}\right), y^{(i)}\right) + \lambda\Omega(\theta)$$

Regularizacioni parametar

Regularizacioni izraz

-
- regularizacioni parametar lambda sluzi za fino podesavanje prilagodljivosti modela tako sto kontroliste nagodbu dva cilja
  - minimizacija gubitka rezultuje prediktivnim modelom, cini da se dobro uklopi u trening podatke
  - minimizacija omega rezultuje jednostavnim modelom. jednostavniji modeli imaju manju varijansu u predikcijama i stavilniji su
- ako je lambda jednaka nuli onda nema regularizacije -> overfitting
- ako je lambda jednaka beskonacnosti onda su parametri teta jednaki nuli ->underfitting

1) Suma apsolutnih vrednosti (L_1 norma): Lasso regression/L_1 regularizacija

$$\Omega = \sum_{d=1}^{D} |\theta_d| = \|\theta\|_1$$

2) Suma kvadrata vrednosti (kvadrirana L_2 norma): Ridge regression/L_2 regularizacija

$$\Omega = \sum_{j=1}^{d} {\theta_j}^2 = \|\theta\|_2^2$$

-

Ridge regression

$$J(\theta) = \frac{1}{2N}\sum_{i=1}^{N}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 + \frac{\lambda}{2}\sum_{i=1}^{d}\theta_j^2$$

RSS: model treba što više da odgovara trening skupu

Magnitude koeficijenata θ treba da su što manje

λ kontroliše nagodbu ova dva cilja

$\lambda = 0$	$\lambda = \infty \Rightarrow \theta = 0$
OLS metod (bez regularizacije)	naš model postaje konstanta $h_\theta(x) = \theta_0$

- zove se ridge (greben) jer multikolinearnost : funkcija cilja nema jedinstveni minimum nego greben pri cemu ce ridge zameniti taj greben minimumom

Priprema podataka za regularizaciju

Podatke je neophodno normalizovati

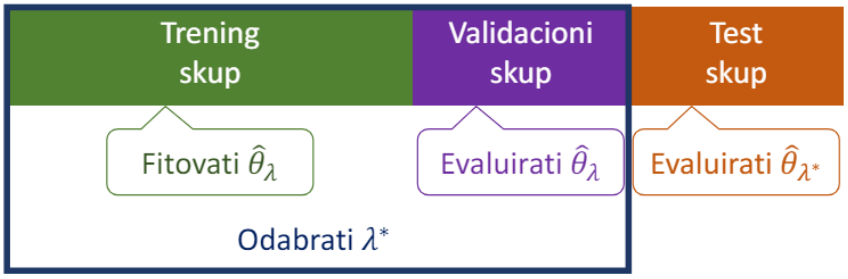
$$x_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

- Neka se sve vrednosti x kreću u istom opsegu $[-1, 1]$
- Uzmimo obeležje x_j i pomnožimo ga se 10^{-6}
- Bez regularizacije: $\theta_j^{\text{new}} = \theta_j^{\text{old}} \cdot 10^6$
- Regularizacija: tretira sva obeležja jednako – praktično bi uticala samo na θ_j

- teta nula je ocekivana vrednost kada su svi ulazi 0, pa nema smisla da forsiramo da bude malo jer to nije indikator prilagodjavanja

- Da ne bismo u jednacinama zasebno tretirali teta 0 mozemo
  - centrirati podatke oko 0
  - primeniti ridge regresiju tretirajuci sve tete jednako
- **malo lambda overfitt, a veliko underfit**

- Podeliti dostupne podatke na:
  - *trening* skup – treniranje modela
  - *validation* (*hold-out*) skup – optimizacija kompleksnosti modela
  - *test* skup – evaluacija performansi modela
- Za svako λ:



ukoliko nemamo dovoljno podataka moramo raditi unakrsnu validaciju, izmesati trening podatke i podeliti ih na k delova, i onda od 1 do k izdvojiti svaki put drugi deo podataka kao validacioni skup i izracunati prosecnu gresku

- postupak unakrsne validacije trebamo ponoviti za svako lambda
- k odabiramo takvo da je k=n - leave one out cross validation
- ali ova procedura je racunarski zahtevna - zahteva fitovanje N modela za svaku razmatranu vrednost lambda
- uglavnom se koristi K=5 ili K=10

- Regularizacija:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} L\left(h_\theta\left(x^{(i)}\right), y^{(i)}\right) + \lambda\Omega(\theta)$$

- L_2 regularizacija/ridge regression:

$$\Omega(\theta) = \sum_{i=1}^{d} \theta_j^2$$

Selekcija obelezja

- najvaznija obelezja treba da imaju najvece koeficijente teta, a obelezja koja nisu u korelaciji sa y treba da se mnoze sa 0
- selektujemo obelezja jer je veliki broj obelezja prilicno zahtevan (osima ako je teta *sparse* - sadrzi veliki broj 0)

- Recimo da je naš model:

$$\theta_1 x_1 + \theta_2 x_2$$

- Recimo da je $x_1 = x_2$
- Model je u tom slučaju

$$(\theta_1 + \theta_2) x_1$$

- Odnosno, da smo inicijalno izbacili x_2 iz modela, težina ispred x_1 bi bila duplo veća i ovo obeležje bi bilo zadržano u modelu

-

- Recimo da imamo skup koreliranih obeležja

- Uzmimo u obzir dva modela:
  1) Jednako rasporediti (male) težine na sva korelirana obeležja
  2) Jednom obeležju dodeliti (veliku) težinu, a težine ostalih postaviti na 0

- Iako su ova dva modela slična (u smislu konačne funkcije koju dobijamo), *ridge* će preferirati rešenje 1)

- U funkciji greške figuriše θ^2 – jedan veliki θ koeficijent bi jako uvećao regularizacioni deo greške

-

- Ako bismo koristili prag da odsečemo neka obeležja eliminisali bismo sva korelirana obeležja

- Iako je jedno od njih (ili ceo skup zajedno) relevantno za zadatak predikcije

- Dakle, *rigde* ne možemo koristiti za selekciju obeležja

$$J(\theta) = \frac{1}{2N} \sum_{i=1}^{N} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_{i=1}^{d} |\theta_j|$$

- Ovo će da dovede do *sparse* modela (neki θ koeficijenti biće postavljeni tačno na 0)

- λ: koliko favorizujemo *sparsity* naspram prilagođavanja podacima

- $\lambda = 0$, vraćamo se na prethodni OLS (bez regularizacije)

- za $\lambda = \infty$, svi koeficijenti θ će težiti 0

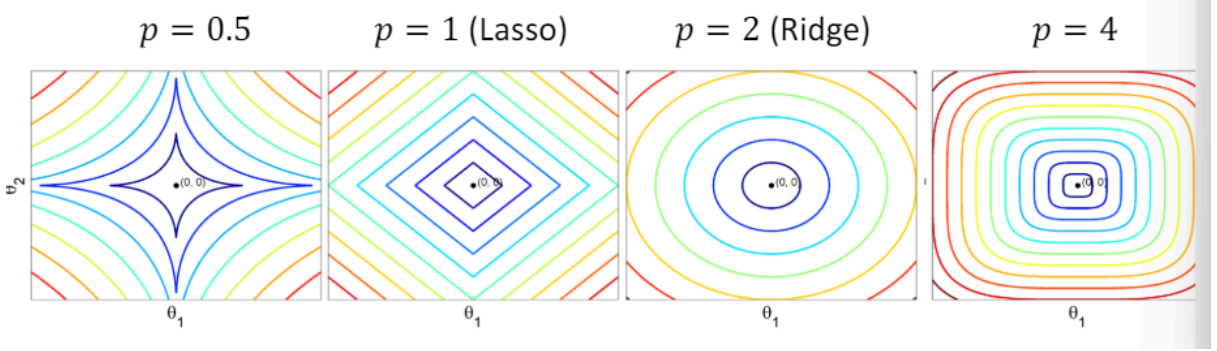
-

- **Ridge se ne moze koristiti za selekciju obelezja a lasso moze**
- **ridge ce dobro raditi kada su stvarni ne 0 koefcijenti mali**

- **ridge nece raditi dobro ako su stvarni ne-0 koeficijenti veliki, ali ce raditi bolje od OLS**

• Lasso (L_1) i Ridge (L_2) su specijalni slučajevi L_p regularizacionog izraza:

$$\left(\sum_d |\theta_d|^p\right)^{\frac{1}{p}}$$



-
- **lasso ima prednost u odnosu na ridge za interpretaciju modela jer postavlja koeficijente na 0**
- **nedostatak lasso je da ciljna funkcija nije diferencijabilna**

Ulaz	• $J(\theta)$ – funkcija koja se optimizuje • θ_0 – početno rešenje • *maxIters* – maksimalan broj iteracija
Postupak	for t = 1, 2,..., *maxIters*: Odabrati koordinatu j $\theta_j^{(t+1)} = \min_{\theta} g(\theta_0^{(t)}, \dots, \theta_{j-1}^{(t)}, \theta, \theta_{j+1}^{(t)}, \dots, \theta_d^{(t)})$
Izlaz	θ (tačka u kojoj funkcija $J(\theta)$ ima minimum)

Coordinate descent za RSS

Ulaz	• $g(\theta)$ – funkcija koja se optimizuje • θ_0 – početno rešenje • *maxIters* – maksimalan broj iteracija
Postupak	for t = 1, 2,…, maxIters: for j=0,1,…,d (round robin) $\theta_j^{(t+1)} = \rho_j$
Izlaz	θ – tačka u kojoj funkcija $J(\theta)$ ima minimum

-stajemo kada nam je korak manji od neke granice epsilon
- lasso odabiramo lambda isto kao i kod ridge train val test ili cross validation
- lasso smanjuje koef u odnosu na OLS tj dobijamo model sa vecim sistemskim odstupanjem i malom varijansom
- sistemsko odstupanje mozemo smanjiti tako sto cemo koristiti lasso za selekciju obelezja i primeniti ols koristeci samo selektovana obelezja

Elastic net

- lasso radi bolje kada model ima mnogo beskorisnih obelezja a ridge obrnuto
- elastic net je linearna kombinacija lasso i ridge cime
  - uklanja ogranicenje na broj selektovanih varijabli
  - ohrabruje grupnu selekciju
  - stabilizuje L1 regularizacioni put
  - a uz pomo lasso vrsi selekciju obelezja

• *Ridge*, LASSO i *Elastic Net* su varijacije istog *penalty term* oblika:

$$P_{alpha} = \sum_{i=1}^{D} \left[\frac{1}{2}(1-\alpha)\theta_i^2 + \alpha|\theta_i| \right]$$

• *Ridge*: $\alpha = 0$
• LASSO: $\alpha = 1$
• *Elastic Net*: $0 < \alpha < 1$

- Specifikacija $P_{alpha} = \sum_{i=1}^{p} \left[\frac{1}{2}(1-\alpha)\theta_j^2 + \alpha|\theta_j|\right]$
  se obično smatra za *naive* Elastic Net

- Na žalost, obično ne radi dobro u praksi – parametri se penalizuju dva puta za isti nivo α (stoga se zove naivan)

- Korekcija:

$$P = (1-\alpha)|\theta|_1 + \alpha\,|\theta|^2 = \lambda_2|\theta|^2 + \lambda_1|\theta|_1$$

  - gde je $\alpha = \frac{\lambda_2}{\lambda_1+\lambda_2}$
  - $\hat{\theta}_{elastic\ net} = (1+\lambda_2)\,\hat{\theta}_{naive\ elastic\ net}$

1NN

- osetljiva na overfitting
- kod 1nn najbliza tacka koja postoji u trening skupu moze imati abnormalno veliku/malu vrednost u odnosu na stvarni trend, pa umesto da uzmemo jednog suseda mozemo uzeti k suseda

KNN

Ulaz	• $T = \{(x^{(i)}, y^{(i)}), i = 1, \dots, N\}$ • $x^{(q)}$ – tačka za koju treba da odredimo $y^{(q)}$ vrednost
Postupak	1. U trening skupu pronaći *k* tačaka najbližih tački $x^{(q)}$: $\{(x^{(NN1)}, y^{(NN1)}), \dots, (x^{(NNk)}, y^{(NNk)})\}$ 2. Predvideti $\hat{y}^{(q)} = \frac{1}{k}(y^{(NN1)} + \dots + y^{(NNk)})$
Izlaz	$\hat{y}^{(q)}$

-
- postoji problem sa diskontinuitetom
- resenje toga je uvodjenje tezinskih faktora udaljenosti, sto znaci pridruziti vecu tezinu opservacijama koje su slicnije nasoj tacki
- dobijamo efekat da kada jedna opservacija izadje iz prozora vidljivosti vrednost se ne menja mnogo
- tezine cemo definisati pomocu jezgra
- isotropic kernel - jezgra koja su funkcija rastojanja bilo koje tacke i nase tacke
- mozemo koristiti gausov kernel kojim odredjujemo koliko su opservacije slicne
- epanchenikov kernel pridruzuje tezine svima za razliku od gausa gde su neke tezine 0

Parametarski pristup

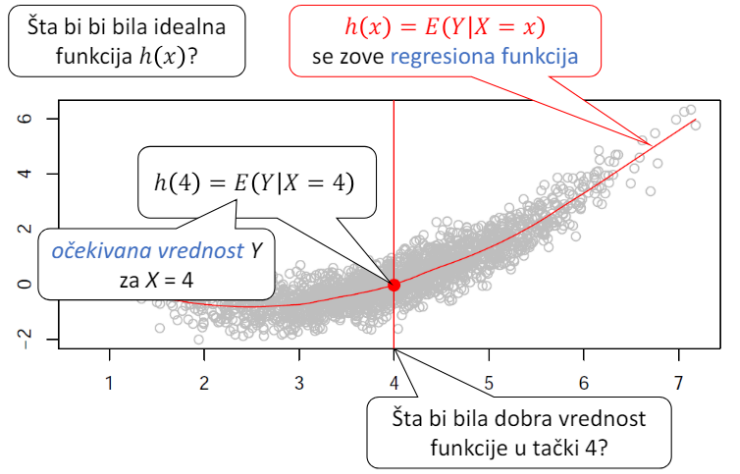
- definisan konacnim brojem fiksnih parametara
- izbor obelezja od velike vaznosti
- za fiksno teta postoji granica koje govori do koje mere je model fleksibilan

Neparametarski pristupi

- izbor obelezja nije kritican problem

- jednostavni su za implementaciju
- fleksibilni
- povecanjem broja podataka povecava se kompleksnost
- jako dobar u praksi ako imamo dovoljno podataka

Regresiona funkcija



Mana procene $h(x) = E(Y|X = x)$

- Najčešće, za određenu tačku x imamo malo (ili čak 0) opservacija za koje važi $X = x$

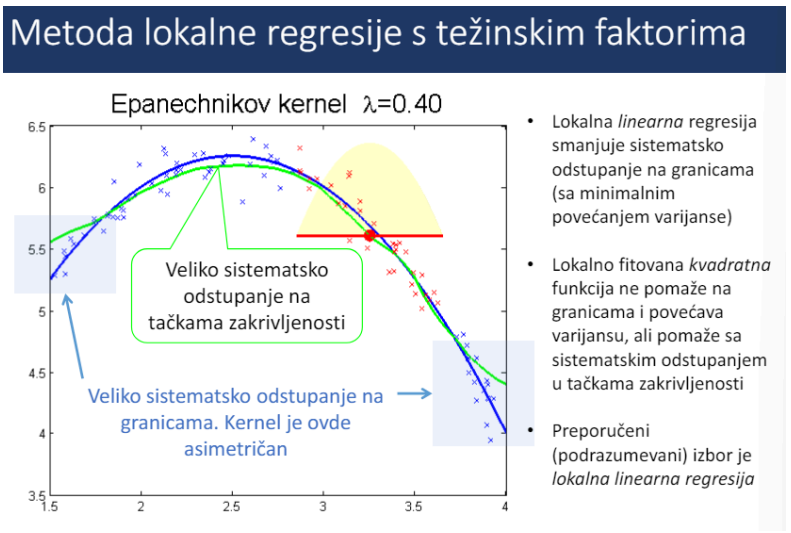
- Posledica je da ne možemo izračunati $E(Y|X = x)$

- Zato ćemo malo relaksirati definiciju:

$$\hat{h}(x) = Ave(Y|X \in \mathcal{N}(x))$$

gde $\mathcal{N}$ označava susedstvo (*neighborhood*) tačke x

- kernel regresija fituje konstantnu funkciju lokalno
  - lokalno -> drugacije za svaku tacku
  - naziva se locally weighted averages
- metodom lokalne regresije sa tezinskim faktorima umesto konstante fitujemo liniju ili polinomijalnu krivu



-
- param metode: linearna regresija bez tezinskih faktora
- neparam metode : knn, kernel regresija lokalna regresija sa tezinskim faktorima
- param metode imamo fiksne parametre teta i nakon njihovog odredjivanja nije nam potreban skup podataka
- neparam prilikom predikcije koristimo trening skup, kolicina toga sto moramo da cuvamo kako bismo reprezentovali hipotezu h raste linearno sa rastom skupa podataka
- ovim dobijamo fleksibilnost i uvodimo sto manje pretpostavki o stvarnoj funkciji ali kompleksnost modela raste sa brojem opservacija
- **fleksibilnost kod neparam** znaci da ako imamo besknoacno podataka bez suma MSE je 0, tj bias i varijansa su nula, kod param to nije moguce jer uvek ima nesto sistemskog odstupanja

- k-nn i kernel rade dobro kada opservacije pokrivaju ulazni prostor, a lose gde opservacije nisu guste
- rade dobro za D<4 i veiko N, tada imamo dovoljno tacka u svakom susedstvu da mozemo izracunati prosek
- ove metode mogu biti lose za veliko D zbog prokletstva dimenzionalnosti - najblizi susedi mogu biti veoma daleko akada imamo veliki broj dimenzija
- -prokletstvo dimenzionalnosti resavamo dodavanjem strukture nasem modelu
- najjednostavnija struktura je linearan model Specificiran je sa D+1 param
- ovakav moel na pati od prokletstva dimeznionalnosti jel se ne oslanja ni na kakva lokalna iskustva, gotovo nikad nije tacan ali sluzi kao dobra i interpretabilna aproksimacije prave funkcije y

Metod maksimalne verodostojnosti

verovatnoca - je da je mis npr tezak 32-34g je 29%
verodostojnost da smo izmerili misa od 34g je 0.12ml

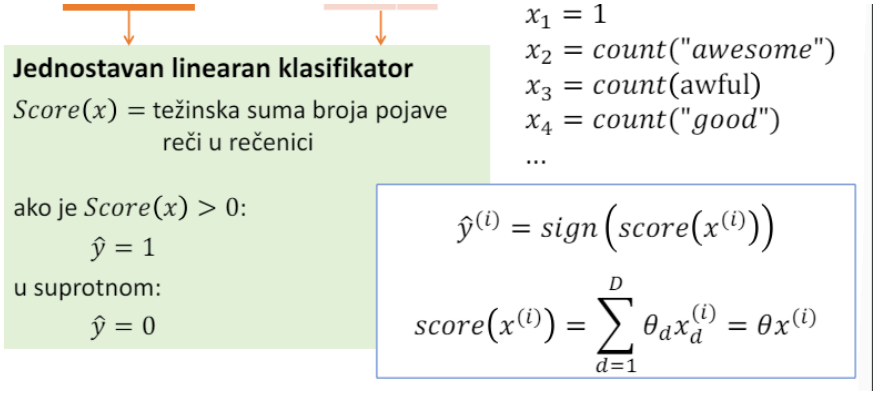
Klasifikacija

Y uzima kao vrednost jednu od konacno mnogo diskretnih kategorija, moze biti binarana i visekategorijska

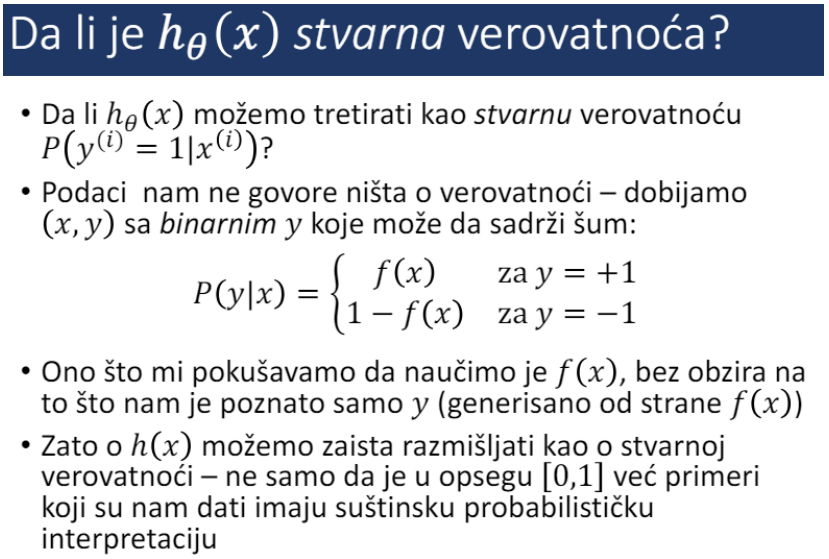
- mozemo primeniti linearnu regresiju za klasifikaciju jer linearna regrsija vraca resenja iz skupa prirodnih brojeva, arecimo binarka klasifikacija vraca -1 ili 1 sto su isto prirodni brojevi

Linearan klasifikator

- svakoj reci dodeljujemo tezinski koeficijent
- koeficijent izrazava koliko pozitivnog/negativnog uticaja ima data rec
- treniranje modela je odredjivanje koeficijenata na osnovu trening skupa
- izlaz je tezinska suma ulaza



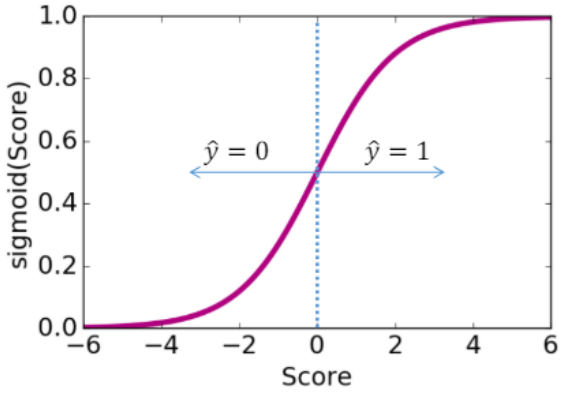
- u slucaju da imamo D obelezja sa ne-nula koeficijentima granica odluke bi bila hiperravan
- prilikom klasifikacije necemo predvidjati samo y^=1 ili y^=0 nego ce klasifikator dati procenu verovatnoce



Logisticka regresija

- specijalan slucaj GLM-a
- kao funkcija g koristi se logistic link function(sigmoid, logit)

$$\sigma(score) = \frac{1}{1 + e^{-score}}$$



-

- Sto je veca magnituda koeficijenata kriva je strmija tj brze postajemo sigurni u predikciju

- Trening skup $\mathrm{T} = \{(x^{(i)}, y^{(i)})\}, i \in \{1, \dots, N\}$
- Svaki pimer opisan je sa *D* obeležja: $x = \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_D \end{bmatrix}, x_0 = 1$
- Ciljna varijabla $y \in \{1,0\}$
- Hipoteza:

$$h_\theta(x) = \sigma(\theta x) = 1/(1 + e^{-\theta x})$$

- Kako odrediti parametre θ?

  - treba da odaberemo teta tako da verovatnoca uocenog trening skupa bude najveca
  - Cilj je naci teta za koje je verodostojnost L(teta) najveca
  - minimalizovacemo negativan logaritam funk verodostojnosti
  - optimizacioni problem koji se javlja je cross entropy error
  - ne postoji closed form solution

- Funkcija L(teta) je konveksna pa ima jedinstveni globalni minimum, za optimizaciju mozemo upotrebiti metod gradijentnog spusta, a obicno se koristi njutnova metoda

- *Update rule*:

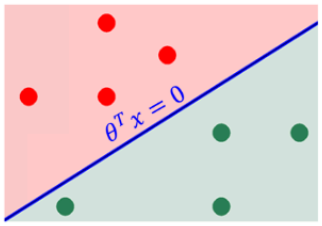
$$\theta_j^{(t+1)} = \theta_j^{(t)} - \alpha \sum_{i=1}^{N} \boxed{\left(h_\theta(x^{(i)}) - y^{(i)}\right) x_j^{(i)}}$$

$\frac{\partial}{\partial \theta} \mathcal{L}(\theta)$

- Izgleda identično linearnoj regresiji! Razlika je samo u obliku hipoteze $h_\theta(x^{(i)})$

- Kao i kod linearne regresije
  - Konvergenciju možemo proveritu iscrtavanjem $\mathcal{L}(\theta)$ kao funcije broja iteracija – $\mathcal{L}(\theta)$ treba da se smanjuje sa svakom iteracijom
  - Bolje je normalizovati obeležja (brža konvergencija)

  - Efekat overfittinga kod klasifikatora
    - koeficijenti teta ce biti velikih magnituda
    - znaci da ce tetax biti velika pozitivna ili velika negativna vrednost pa ce nas model uvek biti veoma blizu 0 ili 1 i postace previse pouzdan u svoje predikcije



- Podaci su linearno separabilni ako postoje koeficijenti θ takvi da:
  - Za sve pozitivne primere važi $\theta X > 0$
  - Za sve negativne primere važi $\theta X < 0$
- Ovo znači da će greška na trening skupu biti 0

- Ako imamo *D* obeležja, linearna separabilnost se dešava u *D*-dimenzionom prostoru
- Sa dovoljno obeležja, podaci će gotovo uvek biti linearno separabilni!

- Efekat linearne separabilnosti na teta
  - maximum liklihood preferira da verovatnoca bude sto veca pa ce koeficijenti teziti da budu sto veci
  - kod linearno separabilnih podataka koeficijenti mogu otici u beskonacnost

Regularizacija

- koristicemo je za penalizovanje velikih vrednosti koeficijenata

- Na $\mathcal{L}(\theta)$ dodaćemo regularizacioni penal:
  - Suma kvadrata koeficijenata (L_2 norma)
    $$\|\theta\|_2^2 = \theta_1^2 + \cdots + \theta_D^2$$
  - Suma apsolutnih vrednosti koeficijenata (L_1 norma)
    $$\|\theta\|_1 = |\theta_1| + \cdots + |\theta_D|$$
  - Prema konvenciji, ne penalizujemo θ_0

Logistička regresija sa L_2 regularizacijom

- Minimizovaćemo sledeću funkciju:
$$J(\theta) = \mathcal{L}(\theta) + \lambda \sum_{j=1}^{N} \theta_j^2$$
- Gradijentni spust:
$$\theta_j^{(t+1)} = \theta_j^{(t)} - \alpha \left[\sum_{i=1}^{N} \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)} + 2\lambda\theta_j \right]$$

- lambda kontrolise nagodbu sistematskog odstupanja i varijanse(kompleksnost modela)
- malo lambda - malo sistematsko odstupanje, valika varijansa
- veliko lambda - veliko sistematsko odstupanje mala varijansa

Logistička regresija sa L_1 regularizacijom

$$J(\theta) = \mathcal{L}(\theta) + \lambda \sum_{j=1}^{N} |\theta_j|$$

- Ova vrsta regularizacije dovodi do **sparse rešenja**
  - Za većinu koeficijenata važi $\theta_j = 0$, a samo za nekolicinu važi $\theta_j \neq 0$
  - Ovim dobijamo na efikasnosti i interpretabilnosti modela

Perceptron

- pristup logisticke regresije modifikovan na taj nacin da je prisiljen da izbaci vrednosti koje su 1 ili -1

$$h(x) = sign\left(\left(\sum_{i=1}^{d} \theta_i x_i\right) - \text{threshold}\right)$$

- Ono što razlikuje jednu hipotezu od druge je naš izbor vrednosti *parametara modela*: θ_i i threshold

-

Ulaz	• $T = \{(x^{(i)}, y^{(i)}), i = 1, \ldots, N\}$  • maxIters
Postupak	for t = 1, 2,..., maxIters: • Iz T na slučajan način odabrati pogrešno klasifikovanu tačku $(x^{(n)}, y^{(n)})$ • Na osnovu $(x^{(n)}, y^{(n)})$ ažurirati parametre θ: $\theta^{(t)} \leftarrow \theta^{(t-1)} + y^{(n)} x^{(n)}$
Izlaz	$h_\theta(x) = sign(\theta^T x)$

PLA azuriranje parametar pokusava da sredi jednu tacko ali pritom moze da pokvari sve ostale tacke jer ih ne uzimamo u obzir prilikom pomeranja granice
ipak ako su podaci linearno separabilin tokom vremena cemo dobiti korektno resenje

Modifikacija perceptrona Pocket

- primenjujemo PLA ali cuvamo najbolju hipotezu, i prekidamo nakon x iteracija, bolji je od perceptrona na podacima koji nisu linearno separabilni

Linearna regresija generalno nnije dobar izbor za klasifikaciju ali moze pomoci pri inicijalizaciji

- problem da za predikcije perceptrona navede probabilisticka interpretacija ili da se perceptron izvede putem metode maksimalne verodostojnosti

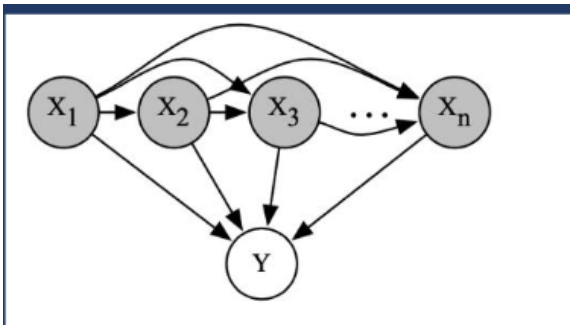
Tipovi klasifikatora

Bazirani na instancama

- KNN
- koriste opservacije direktno bez formiranja modela
- ne uvode pretpostavke o strukturi podataka

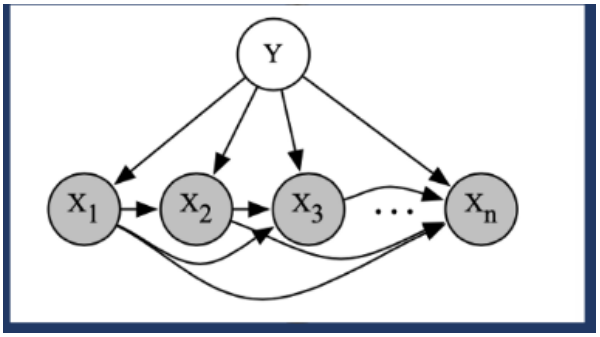
Diskriminativni

- logisticka regresija, svm, boosting
- direktno estimiraju granicu odluke
- Direktno ucimo P(y|x)
    - pretpostaviti formu P(y|x)
    - npr u logistickoj regresiji pretpostavimo sigmoid
- ne mozemo generisati podatke jer ne znamo P(x)



Generativni

- naivni bajes, GMM
- modeluju kakko izgledaju podaci svake klase
- Odredjujemo P(x|y)
    - pretpostaviti formu P(x|y) i P(y)
    - odvojeno modelujemo kako izgleda x jedne i druge klase
    - P(y) - apriorna verovatnoca y
        - Bajesovim pravilom odredjujemo P(y|x)
- Mozemo generisati primere P(x)=Suma P(y)P(x|y)



Poredjenje tacnost

- Generativni modeli su losiji kada pretpostavka o uslovnoj nezavisnosti nije zadovoljena
- Generativni modeli uvode snaznije pretpostavke

- Potrebno je napraviti pretpostavke o raspodeli verovatnoce(a one mogu biti pogresne), ali zato im treba manje podataka za trening
- Pokazalo se da pogotovo u slucaju malih podataka NB cesto pokazuje bolje performanse
    - ignorise interakciju medju atributima pa mu treba manje podataka u odnosu na logisticku regresiju
    - manje verovatno da overfituje ako ima malo podataka

Poredjenje nedostajuce vrednosti

- generativni modelu mogu da rade sa nedostajucim vrednostima, a diskriminativni ne mogu

Poredjenje primena

- generativni modelu mogu da generisu podatke sa istim svojstvima poput podataka na kojima je trenirano
- Diskriminativni modelu mogu da resavaju klasifikacione probleme
- Generativni modeli imaju i ostale primene poput uzorkovanja i generisanja podataka

1vs1 model

- za svaki par klasa c_k i $c_{k'}$ napraviti problem binarne klasifikacije
    - trening podaci sa ck ce biti pozitivni y=1
    - trening podaci sa labelom ck' ce biti negativni y=0
    - zanemariti ostale trening podatke
- Trenirati C(C-1)/2 binarnih klasifikatora (C- broj klasa)
- Predikcija se sastoji iz kombinovanja izlaza rezultujucih klasifikatora

Kada korisititi koji pristup

1 vs all	1vs1
C klasifikatora	C(C-1)/2 klasifikatora
Racunarski manje zahtevan za veliko C	Za treniranje koristimo manji podskup podataka (brze ako koristimo veci podskup)
Izuzetno nebalansirani skupovi podataka za veliko C (znacajno vise negativnih primera)	Manje problematicni sa aspekta neizbal.

Softmax regression model

- generalizacija logisticke regresije za visekategorijsku klasifikaciju
- imamo DxC parametara gde je D br obelezja a C br klasa

- Sigmoidnu funkciju (koju smo koristili u binarnoj logističkoj regresiji):

$$P\left(y = 1|s^{(i)}\right) = \frac{1}{1 + e^{-s(i)}}$$

- ćemo zameniti sa tzv. *softmax* funkcijom:

$$P\left(y = c_j|s^{(i)}\right) = f\left(s^{(i)}\right) = \frac{e^{s(i)}}{\sum_{(k=1)}^{C} e^{s(k)}}$$

- gde je $s = \theta^T x$ („score“), a C broj klasa

Granica odluke: granica izmedju bilo koje dve klasa je prava linija

Nedostajuce vrednosti

- strategije su prociscavanje podataka, procena nedostajucihi vrednosti, modifikacija obucavajuceg algoritma da bude robustan na nedostajuce vrednosti
- prociscavanje podataka - uklanjanje opservacija sa nedostajucim vrednostima ili ukloniiti obelezje za koje mnogim opservacijama nedostaje vrednost
    - prednosti su jednostavno za razumevanje i primenljivo za sve modele
    - mane su uklanjanje opservacija moze da rezultuje odbacivanjem informacija vaznih za resavanje problema, nije jasno uvek kad ukloniti opservaciju a kada obelezje, ove strategije ne pomazu kada treba da damo predikciju za opservaciju sa nedostajucim vrednostima
- procena nedostajucih vrednosti - nedostajucu vrednost zamenimo nasom procenom te vrednosti, npr kategoricko obvelezje zamenimo medijanom poznatih vrednosti tog obelezja,a numericko srednjom vrednoscu ili medijanom poznatih vrednosti tog obelezja
    - naprednije tehnike za to su ocena nedostajuce vrednosti regresijom, knn ili EM
    - prednosti su jednostavno za razumevanje i implementaciju, moze da se primeni na bilo koji model i moze da se koristi prilikom predikcije
    - nedostaci su da moze uzrokovati sistematskom greskom ili dodavanjem sistematskog odstupanja
- modifikacija obucavajuceg algoritma - obucavanje da zna kako da rukuje sa neodstajucim vrednostima
    - prednosti - moze se koristiti i prilikom obucavanja i prilikom predikcije, rezultuje tacnim predikcijama
    - mane - zahteva modifikaciju obucavajuceg algoritma sto moze biti veoma kompleksno, za odredjene algoritme moze biti lako kao decision tree

Naive Bayes

- generativni model, jednostavan linearni klasifikator, probabilisticki model zasnovan na bajesovoj teoremi

---

Aposteriorna verovatnoća | Uslovna verovatnoća | Apriorna verovatnoća

$$P(y = c|x) = \frac{P(x|y = c)P(y = c)}{P(x)}$$

Dokaz

- Interpretacija u kontekstu klasifikacionog problema:
    - Aposteriorna verovatnoća: koja je verovatnoća da konkretan primer pripada klasi, na osnovu uočenih vrednosti obeležja?
-
- **VIDETI PRIMER ZA KANCER SA PREZENTACIJE 7-1**
- apriorna verovatnoca u kontekstu klasifikacije je verovatnoca na cemo naici na odredjenu klasu

- Ako znamo $P(x|y)$ i $P(y)$
- Možemo da damo predikciju klase pomoću Bajesove teoreme:

$$P(y = c|x) = \frac{P(x|y = c)P(y = c)}{P(x)}$$

Dokaz

- Dokaz: verovatnoća da naiđemo na određenu kombinaciju vrednosti obeležja x (bez obzira kojoj klasi pripada)
- $P(x)$ možemo izračunati na sledeći način:

$$P(x) = \sum_{y} P(x, y) = \sum_{c} P(x|y = c)P(y = c)$$

METODA POTPORNIH VEKTORA

- direktan pristup klasifikacionom problemu bez uvodjenja probabilistickog modela
- granica odluke koja razdvaja klase je hiperravan
  - hiperravan je koncept u geometriji kojim se generalizuje koncept
  - deli se D-dimenzioni prostor na dva dela, hiperravan je podprostor D-1 dimenzije
  - $$f(x) = \theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_D x_D = 0$$
- hiperravan je najbolja kad je udaljena od primera obe klase
- Margina razdvajajuce hiperravni je minimum rastojanja od te hiperravni do neke od tacaka skupa podataka
- medju svim hiperravnima koje savrseno razdvajaju podatke treba naci onu sa najvecom marginom
- vektori potpore su tacke najblize hiperravni - kriticne tacke koje odredjuju marginu
- optimizacioni algoritam generise pazrametre teta na taj nacin da samo vektori potpore uticu na njih

Treniranje modela

- kako odabrati hiperravan sa najvecom marginom
  - Hiperravni su predstavljene jednačinom $f(x) = \theta^T x = 0$
  - Uslov: linearna separabilnost – za svako $\left(x^{(i)}, y^{(i)}\right)$ mora da važi $y^{(i)} \cdot f\left(x^{(i)}\right) > 0$
  - Ima beskonačno mnogo θ koji bi ovo zadovoljili
  - Od svih takvih jednačina, mi želimo da nađemo θ tako da $\theta^T x = 0$ bude ravan sa maskimalnom marginom?
  - Naš cilj:
    1. Izraziti marginu kao funkciju parametara θ
    2. Naći θ za koje ta funkcija dostiže maksimum
-

Rešenje SVM optimizacionog problema

- Hipoteza je:

$$f_{\theta,b}(x) = \sum_{i=1}^{N} \alpha_i y^{(i)} x^{(i)} \cdot x + b$$

- $\alpha_i > 0$ samo za vektore potpore
- Klasa se određuje funkcijom $sign\left(f_{\theta,b}(x)\right)$

-

Podaci koji nisu linearno separabilni

- Dozvolićemo greške
- Promena optimizacionog problema:

$$\min \frac{1}{2}\theta^T\theta + C \cdot (\text{br. grešaka})$$

- Dakle, minimizujemo $\theta^T\theta$ plus broj grešaka

- Kako da odaberemo C (*slack penalty*)?
  - Veliko C → jako kažnjavamo greške
  - Malo C → dozvoljavamo mnogo grešaka

- C kontrolise nagodbu izmedju sirine margine i koliko kaznjavamo greske
  - C=besk ->zelimo samo da razdvojimo podatke (overfitting)
  - C=0 -> nije nas briga za greske, ignorisemo podatke (underfitting)
  - Optimalno C dobijamo unakrsnom validacijom ili koriscenjem posebnog validacionog skupa
  - Uvodimo slack variables E, to je kazna za datu gresku i definise se kao rastojanje druge strane margine od date tacke

Osnovna formulacija	Linearno neseparabilan slučaj
$\min \frac{1}{2}\theta^T\theta$	$\min \frac{1}{2}\theta^T\theta + C\left(\sum_{n=1}^{N} \xi_i\right)$
$y^{(i)}\left(\theta^T x^{(i)} + b\right) \geq 1$ za $i = 1,2,\dots,N$	$y^{(i)}\left(\theta^T x^{(i)} + b\right) \geq 1 - \xi_n$ za $i = 1,2,\dots,N$

$$\min \frac{1}{2}\underbrace{\theta^T\theta}_{\text{Margina}} + \underbrace{C}_{\substack{\text{Regularizacioni} \\ \text{parametar}}}\left(\underbrace{\sum_{n=1}^{N} \xi_i}_{\substack{\text{Empirical loss (koliko dobr} \\ \text{fitujemo podatke)}}}\right)$$

SVM koristi hinge loss - 0/1 - ako dobro klasifikujemo 0, ako lose kazna je barem 1, a ako je tacka klasifikovana korektno ali preblizu granice odluke ili pogresno kazna je proporcionalna udaljenosti te tacke od granice odluke

  - kod SVM odluka mora biti linearna
  - a ako problem nije linearno separabilan resavamo ga

- reklasacimjom definicije razdvajanja
- nelinearnim transofrmacijama podatak(kernel trik)

- kernel trik resavanje nelinearno separabilnih podataka dodavanjem jos jedne dimenzije
  - Skup podataka T koji nije linearno separabilan u $\mathbb{R}^d$ može biti linearno separabilan u prostoru $\mathbb{R}^D$ gde je $D > d$
  - Ako imamo transformaciju ϕ koja preslikava skup podataka T u višedimenzioni skup podataka T' koji je linearno separabilan:
    - Možemo da treniramo linearni SVM na skupu podataka T' da bismo pronašli granicu odluke $\vec{\theta}$ koja razdvaja klase u prostoru T'
    - Projektovanje granice odluke $\vec{\theta}$ (linearne u prostoru $\mathbb{R}^D$) nazad u originalni prostor $\mathbb{R}^d$ će rezultovati nelinearnom granicom odluke u $\mathbb{R}^d$
  - Ovo znači da možemo da naučimo nelinearnu granicu odluke koristeći originalnu linearnu formulaciju SVM-a

Originalno rešenje SVM optimizacionog problema:	Zamena skalarnog proizvoda u $\mathbb{R}^N$ sa kernelom k:
$f_{\theta,b}(x) = \sum_{i=1}^{N} \alpha_i y_i x_i \cdot x + b$	$f_{\theta,b}(x) = \sum_{i=1}^{N} \alpha_i y_i k(x_i, x) + b$

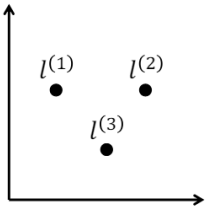
  - Zahvaljujući ovom, **SVM ne mora eksplicitno da radi u više-dimenzionom prostoru** tokom treniranja/testiranja modela
  - Reprezentacije podataka u tom prostoru se ne konstruišu eksplicitno
  - Postoje kerneli koji odgovaraju skalarnim proizvodima u beskonačnodimenzionalnim prostorima, a izračunavaju se efikasno!

1. Koristeći kernel $k(\vec{v}, \vec{w})$ možemo implicitno transformisati skup podataka u višedimenzioni prostor $\mathbb{R}^M$ bez potrebe za dodatnom memorijom i uz minimalan uticaj na vreme izračunavanja
   - Jedini efekat na vreme izračunavanja jeste dodatno vreme za izračunavanje $k(\vec{v}, \vec{w})$. U zavisnosti od k, ovo vreme može biti minimalno
2. Zahvaljujući 1. možemo **efikasno** naučiti nelinearne granice odluke pomoću SVM-a, jednostavnom **zamenom sklarnih proizvoda vektora u SVM optimizaciji sa $k(\vec{v}, \vec{w})$**

- *Kernel funkcije* $k(\vec{v}, \vec{w})$ su funkcije, koje:
  - za data dva vektora v i w u prostoru $\mathbb{R}^N$
  - mogu implicitno da izračunaju skalarni proizvod v i w u prostoru $\mathbb{R}^M$
  - bez eksplicitne transformacije v i w u prostor $\mathbb{R}^M$
- Jednostavnije: kernel je skalarni proizvod u nekom drugom prostoru

-

Šta je kernel?



- Odabrati tačke $l^{(1)}$, $l^{(2)}$ i $l^{(3)}$
- Ove tačke ćemo zvati *landmarks*
- Definisaćemo meru sličnosti kao:

Euklidska udaljenost

$$\text{similarity}(a, b) = \exp\left\{-\frac{\|a-b\|^2}{2\sigma^2}\right\}$$

Kernel (ova konkretna mera je RBF (Gausov) kernel)

-

- u gausovom kernelu o diktira koliko ce brzo opadati slicnost sa rastojanjem x i njega treba da odredimo iz podatka

kako odabrati landmarks

- na mesto svake instance iz trening skupa postavicemo jedan landmark
- na ovaj nacin svako obelezje meri koliko je novi primer udaljen od primera uocenih na trening skupu, broj obelezja je N

SVM sa kernelom sumarizacija

1. Dat je trening skup $T = \{(x^{(1)}, y^{(1)}), \dots, (x^{(N)}, y^{(N)})\}$. Odabrati *landmarks* tako da:
$$l^{(1)} = x^{(1)}, l^{(2)} = x^{(2)}, \dots, l^{(N)} = x^{(N)}$$
2. Svaki primer $x^{(i)}$ iz trening skupa predstaviti putem novih obeležja:
$$f_1 = k(x^{(i)}, l^{(1)})$$
$$f_2 = k(x^{(i)}, l^{(2)})$$
$$\dots$$
$$f_N = k(x^{(i)}, l^{(N)})$$
$$f^{(i)} = \begin{bmatrix} f_0 \\ f_1^{(i)} \\ \dots \\ f_N^{(i)} \end{bmatrix}, f_0 = 1$$
3. Trenirati SVM koristeći trening skup $\{(f^{(1)}, y^{(1)}), \dots, (f^{(N)}, y^{(N)})\}$

-

Popularni kerneli

- linearni kernel je razuman ako imamo veliko d a malo n i ne zelimo kompleksne granice odluke u visedimenzionom prostoru jer imamom mali broj primera pa se lako moze desiti da overfittujemo
- RBF (radial basis function, gausov) - podesavamo parametar o^2, razuman izbor ako imamo malo D a veliko N, **skaliranje obelezja** jer se racuna eukliska distanca
- polinomijalni
- sigmoid kernerl

- odredjivanje parametara kernela postupkom unakrsne validacije

SVM PARAMETRI

- parametar C - penalizacija
- kernel
- parametri kernela

unakrsna validacija ili koriscenje posebnog validacionog skupa

Logisticka regresija vs SVM

- ako je D veliko u poredjenju sa N
    - nemamo dovoljno podataka za fitovanje kompleksnih granica
    - koristiti ili LR ili SVM bez kernela(linear)
- ako je D malo a N srednje velicne
    - SVM sa RBF kernelom
- ako je D malo a N veliko
    - LR uz manuelno dodavanje obelezja ili SVM bez kernela(inace ce biti spor)
- ako zelimo estimaciju verovatnoce LR
- LR i SVM bez kernela daju slicne performanse

Osobine SVM

- fleksibilan prilikom izbora funk slicnosti
- Sparsness resenja kada se radi sa velikim skupom ppodtaka, granica odluke je odredjena samo podskupom podataka
- velika dimenzionalnost nije problem
- overfitting se kontrolise sa soft margine pristupom
- jednostavan konveksan optimizacion alg i garantovano da konvergira u globalni min

Primene

- kategorizacija teksta
- object detection
- rucno pisane cifre
- dijagnoza raka

Stacking

- ideja je da naucimo model da li su modeli dobro nauceni

Ulaz	• $T = \{(x^{(i)}, y^{(i)}), i = 1, \dots, N\}$ • Modeli 1. nivoa $L_1, L_2, \dots, L_M$ • Model 2. nivoa L
Postupak	1. Trenirati modele 1. nivoa: • for $m = 1 \dots M$: $h_m = L_m(T)$ 2. Trenirati meta-model L: • $T' = \{\}$ (generišemo novi treing skup za L) • for $i = 1 \dots N$: for $m = 1 \dots M$: $\hat{y}_m^{(i)} = h_m(x^{(i)})$ $T' = T' \cup \{(\hat{y}_1^{(i)}, \hat{y}_2^{(i)}, \dots, \hat{y}_M^{(i)}), y^{(i)}\}$ • $h' = L(T')$
Izlaz	$g(x) = h'(h_1(x), h_2(x), \dots, h_M(x))$

Before the fact/After the fact

- before the fact - resenja se razvijaju sa ciljem da budu kombinovana
    - ne maksimizujemo performanse individualnih klasifikatora - svaki klasifikator se kreira tako da bude dobar deo mesavine
    - bagging boosting
- after the fact - kombinuje vec postojeca resenja
    - resenja su razvijana nezavisno, maksimizovane su performanse svakog klasifikatora posebno

- dobro je da koristimo ansamble jer smanjujemo rizik odabira loseg klasifikatora
- uslovi za uspesan ansambl su dva neophodna i dovoljna uslova da bi ansambl klasifikatora imao vece performanse od svih svojih individualnih clanova su
    - klasifikatori moraju biti tacni
    - klasifikatorij moraju biti medjusobno razliciti
- previse podataka
    - ako je kolicina podataka prevelika treniranje jednog klasifikatora moze trajati dugo
    - resenje je da se trenira vise razlicitih klasifikatora na razlicitim delovima podataka
- premalo podataka
    - smanjuju varijansu i moguci overfitting
- prednosti ansambla
    - redukuju varijansu
    - reduju sistematsko odstupanje

Jednostavni slabi prediktori

- mala varijansa(stabilni) i nisu skloni overfittingu
- ima problem sa velikim sistematskim odstupanjem i ne moze da resi slozene probleme

Poboljsavanje slabog prediktora

- dodavanje obelezja (npr polomijalna obelezja u slucaju log regresija i pokusati izbeci overfitting)
- kombinovanje vise slabih prediktora

Model ansambla - obucavanje vise modela da rese isti problem i kombinovati njihove glasove

- Kako kombinovati glasove - regresijom(prosek), klasifikacijom (glasanje) ili stacking

Kako obuciti vise razlicitih modela

- manipulacija obucavajuceg algoritma
  - na istim podacim trenirati razlicite modele nezavisno, a mozemo i koristiti iste modele sa razlicitim parametrima
- manipulacija trening skupom
  - napraviti vise razlicitih trening skupova uzorkovanjem
- manipulacija skupa obelezja
  - isti podaci ali razliciti podskupovi obelezja (random forest)
- manipulacija klasnih oznaka
  - 1vs1
  - 1vsAll
  - Veliki broj klasa, generisati L binarnih particija i trenirati L binarnih klasifikatora, klasifikacija: klasa cije particije dobiju najvise glasova

Bagging

- manipulacija trening skupom treniranje istog modela na razlicitim podacima

Ulaz	• $T = \{(x^{(i)}, y^{(i)}), i = 1, \dots, N\}$ – trening skup • M – broj slabih prediktora • Osnovni model L (korišćen za obuku pojedinačnih članova ansambla)
Postupak	• for $m = 1, \dots, M$ • Napraviti podskup skupa podataka T_m uzorkovanjem N primera sa zamenom iz T • Obučiti model $h_m = L(T_m)$
Izlaz	$\hat{y}(x) = \frac{1}{M} \sum_{m=1}^{M} h_m(x)$

svaki podskup ™ ima isti broj primera N kao i originalni skup T ali neki primeri su izostavljeni dok se drugi ponavljaju i bude oko 63.2% originalnih primera u instanci

- prednosti bagginga da redukuje varijansu i pomaze da izbegnemo overfitting, izuzetno koristan u slucaja kada imamo modele sa velikom varijansom i sumom, i manje je oseetljiv na relevantna obelezja

Boosting

- umesto obucavanja jednog klasifikatora
- obucicemo mnogo lsabih koji su dobri na razlicitim delovima ulaznog prosotra
- Kako prinudiiti klasifikatora da nauce razlicite delove ulaznog prostora - fokusiracemo se na primere na kojima su prethodni klasifikatori pogresili
- kako uvecati uticaj odredjenog primer? svakom primeru u trening skupu cemo dati odredjenu tezinu
- Ucenje na oteznjenim podacima
  - ponovo uzorkovati skup podataka i na slucajan nacin izabrati N instanci pri cemu tezi primeri imaju vecu verovatnocu selekcije i mogu biti selektovani vise puta

- koristiti obucavajuci algoritam koji moze direktno da koristi tezine primera kao sto su LR i NB
- boosting je pohlepan algoritam jer u svakom koraku bira lokalno optimalno resenje, odluku donosi inkrementalno u malim koracima, svaki korak v odi ka popravljanju trenutnog stanja i blizi je cilju ali se ne vodi racuna o posledicama losih izbora

AdaBoost

Ulaz	• $T = \{(x^{(i)}, y^{(i)}), i = 1, \dots, N, y \in \{-1, +1\}\}$ – trening skup • M – broj slabih prediktora ?
Postupak	• Inicijalizovati težine primera tako da budu uniformne $\alpha_i = \frac{1}{N}, i = 1, \dots, N$ • for $m = 1, \dots, M$ • Obučiti klasifikator $h_m(x) \to \{+1, -1\}$ koristeći težine α_i • Odrediti težinu klasifikatora w_m ? • Rekalkulisti težine instanci α_i
Izlaz	$\hat{y} = \text{sign}\left(\sum_{m=1}^{M} w_m h_m(x)\right)$

Kako odredjujemo koeficijente Wm
kako merimo gresku na oteznjenim podacima - ako smo dodelili vecu tezinu nekim primerima zelimo klasifikator koji dovbro klasifikuje te primere, za gresku cemo uzeti oteznjen broj pogresno klasifikovnih primera

- Ukupna težina grešaka:

$$\sum_{i=1}^{N} \alpha_i \mathbb{I}(h_m(x^{(i)}) \neq y^{(i)})$$

- Ukupna težina instanci:

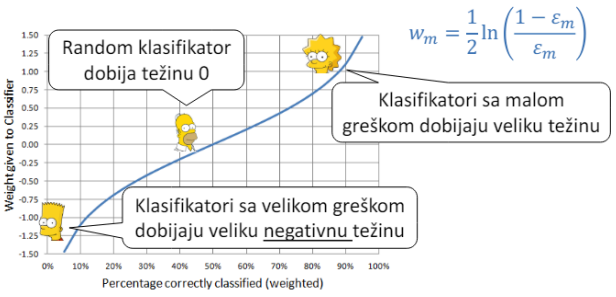
$$\sum_{i=1}^{N} \alpha_i$$

- Otežinjena greška:

$$\varepsilon = \frac{\text{ukupna težina grešaka}}{\text{ukupna težina instanci}} \in [0, 1]$$

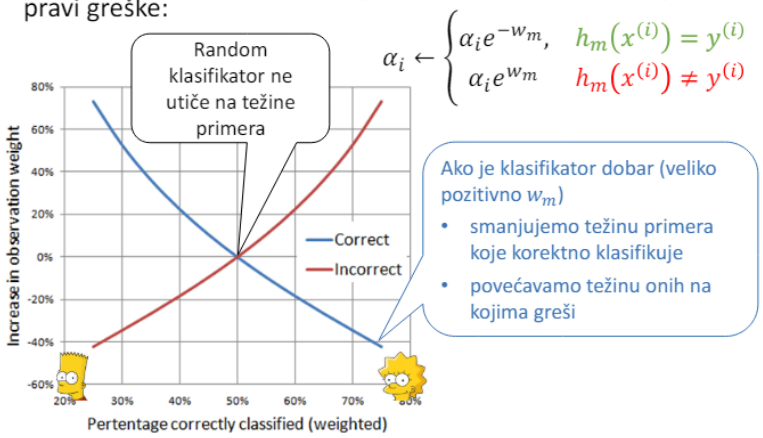
  - Najbolja vrednost ε je 0
  - Najgora vrednost ε je 0.5

- AdaBoost: računanje koeficijenta w_m za klasifikator $h_m(x)$ za koji je otežinjena greška ε_m



Problem 2: kako odrediti težine primera α_i?

- AdaBoost: ažuriranje težina α_i bazirano na tome gde $h_m(x)$ pravi greške:



Ako je primer bas tezak mnogi klasifikatori grese na njemu pa mu tezina postaje bas velika isto vazi i obrnuto za lake primere. To dovodi do numericke nestabilnosti , resenje je da u svakoj iteraciji normalizujemo tezine primera da se sabiraju na 1

AdaBoost sumarizacija

Ulaz
- $\{(x^{(i)}, y^{(i)}), i = 1, \dots, N, y \in \{-1, +1\}\}$ – trening skup
- M – broj slabih prediktora ?

Postupak
- Inicijalizovati težine primera tako da budu uniformne $\alpha_i = \frac{1}{N}, i = 1, \dots, N$
- for $m = 1, \dots, M$
  - Obučiti klasifikator $h_m(x)$ koristeći težine α_i
  - Izračunati koeficijent w_m → $w_m = \frac{1}{2}\ln\left(\frac{1-\varepsilon_m}{\varepsilon_m}\right)$
  - Rekalkulisti težine α_i → $\alpha_i \leftarrow \begin{cases} \alpha_i e^{-w_m}, & h_m(x_i) = y_i \\ \alpha_i e^{w_m} & h_m(x_i) \neq y_i \end{cases}$
  - Normalizovati težine α_i → $\alpha_i \leftarrow \frac{\alpha_i}{\sum_{j=1}^{N} \alpha_j}$

Izlaz

$$\hat{y} = \text{sign}\left(\sum_{t=1}^{T} w_t f_t(x)\right)$$

Adaboost

- dizajniran za binarku klasifikaciju, moze se koristiti i za regresiju
- moze se koristiti za boosting performansi ml algoritma, najbolje je koristiti ga sa slabim prediktorima
- algoritam koji najvise odgovara za adaboost je stablo odlucivanja sa jednim nivoom dubine

Kvalitetni podaci

- adaboost se stalno prilagodjava pogresno klasifikovanim primerima, zbog toga moramo da pazimo na kvalitet podataka, ima problema sa sumom
- outlieri
  - dobra strana je da boosting moze da ih identifikuje jer se fokusira na primere teske za klasifikaciju
  - previse outliera moze da redukuje performanse i poveca vreme potrebno za konvergenciju

Bagging vs Boosting

- povezanost pojedinacnih modela
    - bagging - modeli nezavisni
    - boosting - svaki model se gradi na osnovu prethodnog
- predikcija finalnog modela
    - bagging - jednaki glasovi
    - boosting - otezinjeno glasanje
- efikasnost
    - bagging jednostavniji a nezavisnost modela cini da ih mozemo obucavati paralelno
- Bagging redukuje varijansu osnovnog modela, pogodan je za modele sa velikom varijansom a malim sys odstupanjem (kompleksne modele)
- Boosting moze da redukuje i varijansu i sys odstupanje - u boostingu se trening greska stabilno smanjuje, pogodan je za slabe klasifikatore niske varijanse a velikog sys odstupanja
- Boosting daje bolje performanse
- Sklonost overfittingu
    - bagging resava problem overfittinga
    - boosting je robusniji u odnosu na osnovni model ali nije imun na overfitting
    - bagging je obicno efektivniji od boostinga ako nemamo veliko sistematsko odstupanje i samo zelimo da smanjimo varijansu
- bagging efektivniji od boostinga ako podaci imaju sum ili outliere

KLASTEROVANJE

Sta je klasterovanje

- za dati skup opservacija sa definisanom merom udaljenosti izmedju opservacija, grupisati opservacije u odredjeni broj klastera tako da clanovi unutar jednog klaster budu sto blize clanovima istog klastera i budu sto dalji od clanova drugih klastera
- klasterovanje je nenadgledano obucavanje, podaci nisu anotirani i potrebno je da pronadjemo strukturu posmatrajuci samo ulaz x
- klastere definisemo putem centroida i oblika/rasprostiranja oko centroida
- opservaciju dodeljujemo klasteru ako je score pripadanja tom klasteru veci od score pripadanja ostalim klasterima, cesto se zbog jednostavnosti racuna rastojanje od centra klastera do opservacije

- zadatak klasterovanja deluje tezak ali postoje dve stvafi koje nam omogucavaju da to uradimo
    - struktura koja postoji u samim podacima
    - definicija sta je klaster(koju strukturu pokusavamo da pronadjemo u podacima npr elipsoidni klasteri)

K-means

Ulaz	• $T = \{x^{(i)}, i = 1, \dots, N\}$ – trening skup • K – broj klastera **Kako da odredimo *K*?**
Postupak	• Na slučajan način inicijalizovati centroide $\mu_1, \dots, \mu_K$ • Dok ne dođe do konvergencije: **Da li konvergira?** • Za svaku opservaciju $x^{(i)}$ pronaći klaster sa najbližim centroidom (i dodeliti opservaciju tom klasteru): **Šta znači „blizu“?** $z^{(i)} \leftarrow \arg\min_j \|\mu_j - x^{(i)}\|_2^2$ • Ažurirati centroide klastera prema opservacijama koje su im dodeljene (*mean*) $\mu_j = \frac{1}{n_j} \sum_{i:z^{(i)}=j} x^{(i)}$ n_j - broj opservacija u klasteru j
Izlaz	• $\{(x^{(i)}, z^{(i)}), \ i = 1, \dots, N\}$ **Da li smo dobili kvalitetno rešenje?**

DA LI KONVERGIRA

- K-means alternira između dva koraka:
  1. Dodeli opservacije najbližem centroidu
  $$z^{(i)} \leftarrow \arg\min_j \|\mu_j - x^{(i)}\|_2^2$$
  2. Ažuriraj centroide da budu srednja vrednost dodeljenih opservacija
  $$\mu_j = \frac{1}{n_j} \sum_{i:z^{(i)}=j} x^{(i)} : \ \mu_j \leftarrow \arg\min_{\mu} \sum_{i:z^{(i)}=j} \|\mu - x^{(i)}\|_2^2$$

- Alteriranje minimizacije
  1. Minimizacija J promenom z za fiksno μ
  2. Minimizacija J promenom μ za fiksno z

  → *coordinate descent* → Vrednost J će se monotono smanjivati do konvergencije

Dakle sigurno konvergira

- kako ciljna fukncija nije konvergrentna ne mozemo da garantujemo globalni optimum
- rezultati k means zavisi od inicijalizacije
  - resenje ovoga je ili vise slucajnih inicijalizacija (vrati najbolje resenje)
  - ili kmeans ++
    1. odabrati prvi centroid tako da bude jedna od opservacija
    2. za svaku od opservacija izracunati rastojanje od najblizeg centroida
    3. odabrati novi centroid kao jednu od tacaka skupa podataka
    4. ponavljati 2 i 3 dok ne odaberemo k centroida

KAKO DA PROCENIMO KVALITET DOVBIJENIH KLASTERA

- za fiksan broj klastera Kmeans ima za cilj da minimizuje sumu kvadratnih rastojanja opservacija od centroida
- to mozemo koristiti kao meru evaluacije
- Ako uvecamo broj klastera mozemo da rafiniramo klastere sve vise i vise prema podacima do ekstremnog slucaja K=N gde je svaka tacka poseban klaster. Heterogenost je 0
- Sa povecanjem K smanjuje se heterogenost

KAKO DA ODREDIMO BROJ KLASTERA

- heuristika lakat na krivoj
    - metod je i racunski zahtevan, treba da crtamo najmanju mogucu heterogenost sto znaci da bismo za svaku vrednost K trebali isprobati sve moguce podele na K klastera

Kako da odaberemo K?

- imajte u vidu da je lakat na krivoj samo heuristika
- zavisi od toga sta hocemo da klasterujemo
- ne postoji zlatno pravilo nego sve zavisi od domena i zadatka

KAKO DEFINISATI METRIKU UDALJENOSTI/SLICNOSTI

- E

GMM

- obucavanje pomocu expectation maximization
- ako znamo parametre klastera lako mozemo odrediti meke dodele opservacije klasterima
- ako znamo tvrde pripadnosti klasteru mozemo da odredimo parametre klastera
- estimiracemo parametre na osnovu podataka iz klastera pomocu metode maksimalne verodostojnosti
- dakle cak iako znamo meke pripadnosti klasterima mozemo da odredimo parametre klastera

EM METODA

- E korak procena odgovornosti klastera na osnovu estimiranih parametara klastera
- M korak odrediti parametre klastera maksimizacijom verodostojnosti podataka
- EM je coordinate ascent algoritam, E i M koraci se mogu povezati sa alternirajucim maksimizacijama ciljne ufnkcije
- Konverigra u loklani optimum
- Konvergenciju mozemo proveriti iscrtavanjem vrednosti algoritma verodostojnosti podataka u svakoj iteraciji
- inicijalizacija - random, kmeans++,kmeans mesh podela
- podlozna overfittingu, mnogo veci problem ako imamo vise dimenzija
- najcesce se overfitting deava zbog underflow problema resenje je da ne dopustimo da varijanse padnu na 0
- alternativni bajesovski pristup gde se na parametre klastera dodaje prior

GMM VS KMEANS

- EM sa beskonacno malim varijansama = kmeans
- kmeans je specijalan slucaj EM
- EM je fleksibilniji i ima deskriptivniji izlaz ali ima mnogo vise param i racunarski je zahtevniji

GMM je probabilisticki pristup klasterovanju

Ogranicenja kmeans

- cvrsta dodela objekata klasteru bez kvantifikacije nesigurnosti
- pretpostavlja sfericno simetricne klastere

Model mesavina

- omogucava meku dodelu objekata klasterima
- pored centra uzima u obzir i oblik

- omogucava da ucimo tezine razl dimenzija

Multivariate Gaussian 2D vise od jedne varijable dakle dodeljujemo gustinu veerovatnoce za svaku mogucu kombinaciju vrednosti varijabli

Kombinacija oteznjenih Gausijana

- svakoj klasi odgovara jedna gausova komponenta kojoj cemo dodeliti tezinu koja odgovara porporciji date klase u bodacima

PCA

- ideja je kreirati novi podskup obelezja koji dobro sumarizuje polazna obelezja, znaci odabrati podskup takav da su obelezja relevantna za ciljnu funkciju f
- zasto zelimo manje dimenizja
  - kompresija manje zauzece memorije i diska, znacajno za ubrzanje algoritma
  - uklanjanje suma (irelevatnih obelezja) koji mogu da degradiraju performanse
  - vizuelizacija - bolje razumevanje podataka proizvodi izgradnju boljih modela
- smanjicemo broj dimenzija primenom neke transofrmacije
  - idealno bismo dobili kao rezultat jedno obelezje koje bi bilo sama ciljna funkcija
- ako mozemo da smanjimo dimenzionalnost bez da povredimo trening skup poboljsali smo generalnizaciju
- kako da smanjimo broj dimenzija
  - selekcija obelezja
  - redukcija dimenzionalnosti tj kreirati nova obelezja koja predstavljaju kombinaciju starih obelezja
- PCA konstruise mali broj linearnih obelezja koja sumarizuju ulazne podatke
- Cilj Pca je da konstruise intuitivniju bazu gde je vecina koordinata mala
- male koordinate tretiramo kao slucajne fluktuacije i postavljamo ih na 0
- PCA pronalazi koordinatni sistem koji minimizuje ukupnun gresku rekonstrukcije
- PCA: cilj je zadrzati sto vise informacija u podacima -> novi koordinatni sistem odredjujemo tako da zadrzimo sto je moguce vise varijabilnosti u projekcieji
  - Ako pretpostavimo da podaci imaju srednju vrednost 0, matrica

$$\Sigma = \frac{1}{N}\sum_{i=1}^{N} x^{(i)}\left(x^{(i)}\right)^{\mathrm{T}}$$

  - predstavlja empirijsku matricu kovarijanse podataka

- Dakle, PCA možemo primeniti:
  - izračunavanjem sopstvenih vrednosti matrice kovarijanse $\Sigma = X^T X$ ili
  - primenom SVD metode direktno na matricu podataka X

- Prednosti SVD:
  - numerička stabilnost (računanje $X^T X$ može da izazove gubitak preciznosti)
  - Ako je $D \gg N$ (više obeležja nego instanci), $\Sigma \in R^{D \times D}$ može biti veoma velika matrica

Pretprocesiranje podataka za pca

- podaci moraju biti centrirani
- opciono normalizaovati podatke

Ulaz	• $X \in \mathbb{R}^{N \times D}$ – matrica trening podataka (u redovima se nalaze N instanci, a u kolonama D obeležja) • K $(0 < K \leq D)$
Postupak	1. Centrirati i (opciono) normalizovati X 2. Primeniti SVD na X: $[U, S, V] = \text{svd}(X)$ 3. Neka su $V_K = [v_1, v_2, \ldots, v_K]$ prvih K kolona matrice V 4. Matrica sa novim (PCA) obeležjima je: $Z = XV_K$, a rekonstrukcija je $\hat{X} = XV_K V_K^T$
Izlaz	$Z \in \mathbb{R}^{N \times K}$

-
- obelezja dobijena pomocu pca metode su linearne kombinacije originalnih obelezja
- svi koraci primenjeni u pca su nenadgledani
- glavne komponente kumulativno objasnjavaju varijansu u podacima

Lose upotrebe pca

- manje obelezja - manje sanse za overfitting, bolje koristiti regularizaciju
- prvo probati primenu algoritma bez pca

- Ručno konstruisana obeležja: simetriju i intenzitet
- Slično ovim obeležjima, dva obeležja dobijena pomoću PCA omogućavaju prilično jasno razlikovanje klasa
- Glavna prednost PCA obeležja nad ručno konstruisanim obeležjima jeste što su ova automatski generisana – ne moramo znati ništa o problemu prepoznavanja cifara kako bismo ih dobili
- Ovo je takođe njihov najveći nedostatak – dobijena obeležja su dokazano dobra da rekonstruišu originalne ulazne podatke, ali, nema garancije da će biti korisna prilikom rešavanja problema koji pokušavamo da rešimo
- U praksi, korišćenje domenskog znanja za konstrukciju obeležja, zajedno sa automatskim metodama za redukciju dimenzionalnosti obično ima najbolje performanse

NEDOSTACI PCA

- nema garancija da ce biti unapredjenja
- pca ne koristi info o klasnom obelezju
- LDA je nadgledan metod koji istovremeno redukuje dimenzionalnost i vrsi klasifikaciju

- nedostaci pca su i linearnost, on moze da kreira samo linearna obelezja
- • Kernel-PCA: način da se kombinuje PCA sa nelinearnim transformacijama obeležja, bez da moramo posetiti Z prostor (slično kernel triku kod SVM)

Prakticni saveti za primenu ml

1. Dijagnostika obucavajuceg algoritma

Slepo isprobavanje moze da upali ali zahteva mnogo vremena, zato je bolji pristup izvrsiti dijagnostiku. ML diagnostics je skup tehnika kooje mozemo primeniti u cilju dobijanja uvida u to sta vezano za obucavajuci algoritam radi ili ne radi

2. Analiza gresaka modela
3. Kako zapocetu rad na problemu ML

Informativnost obelezja - verifikacija pomocu eksperta
Strategija
Preporucen pristup resavanju problema

1. zapoceti sa jednostavnim algoritmom
2. iscratati learning curves da odlucimo oko selekcije obelezja itd
3. Analiza gresakam odela

- Procena kvaliteta modela se bavi ocenom greske predvidjanja modela
- selekcija modela se bavi izborom jednog od vise mogucih modela
- zasnivaju se na merama kvaliteta i tehikama evaluacije i izbora

Mere kvaliteta za klafikaciju

- accuracy
  - true positives/all
  - nije dobar izbor u slucaju neizbalasniranih kategorija (gde jednih ima vise od ostalih)
  - tabela kontigencije

		Tačna klasa	
Predikcija		1	0
	1	tp	fp
	0	fn	tn

- Obično se za $y = 1$ uzima ređa klasa
- **tp** (*true positive*): broj instanci za koje je model predvideo $y = 1$ i stvarna klasa tih instanci je zaista 1
- **fp** (*false positive*): broj instanci za koje je model predvideo $y = 1$, a stvarna klasa tih instanci je 0
- **tn** (*true negative*): broj instanci za koje je model predvideo $y = 0$ i stvarna klasa tih instanci je 0
- **fn** (*false negative*): broj instanci za koje je model predvideo $y = 0$, a stvarna klasa tih instanci je 1

- F1
  - 2*(precision*recall)/(precision+recall)
- precision/recall
  - preciznost je tp/(tp+fp)
  - recall tp/(tp+fn)
  - dobar model ima i veliku preciznos i veliki odziv

1. Predviđamo $\hat{y} = 1$ samo ako smo sigurni u predikciju

$$\hat{y} = \begin{cases} 1, & h_\theta \geq 0.7 \\ 0, & h_\theta < 0.7 \end{cases}$$

- Izbegavamo fp – naći ćemo malo pacijenata sa rakom, ali smo za te pacijente sigurni da imaju rak
- Veća preciznost
- Manji odziv

2. Izbegavamo da promašujemo previše slučajeva raka

$$\hat{y} = \begin{cases} 1, & h_\theta \geq 0.3 \\ 0, & h_\theta < 0.3 \end{cases}$$

- Izbegavamo *fn* – naći ćemo gotovo sve pacijente sa rakom, ali ćemo za dosta njih koji nemaju rak (pogrešno) predvideti da ga imaju
- Manja preciznost
- Veći odziv

-

Mere kvaliteta za regresiju

- rmse
  - koren srednjekvadratne greske
- r^2
  - koef determinacije - meri udeo varijanse ciljne promenljive koji je objasnjen modelom

Tehnike evaluacije

- podaci korisceni za procenu kvaliteta ne smeju biti upotrebljivani prilikom treninga
- treninig i test treba da imaju istu raspodelu

Prilikom stratifikacije i samog deljenja podataka obezbediti da delovi imaju istu raspodelu kao i ceo skup podataka, sto je tesko za male skupove podataka

TRI PRINCIPA UCENJA

- Okamova ostrica - simbol principa
  - imamo objasnjenje podataka, nastavljamo da orezujemo objasnjenje sve dok ne stignemo do minimuma koji je i dalje konzistentan sa podacim, i njega smatramo najboljim mogucim objasnjenjem
  - najjednostavniji model koji odgoara podacima je najverodostnojinji
  - sta znaci da je model jednostavan
    1. KOMPLEKSNOST POJEDINACNE HIPOTEZE H
       - mere se odnose na pojedinacan objekad i kompleksnost je svojstvo samog objekta
       - MDL min description length - dati objekat pokusavamo da specificiramo sa najmanje moguce bitova
    2. KOMPLESKONST SKUPA HIPOTEZA H
       - kompleksnost se izracunama za skup objekata
       - entropija
       - VC dimenzija - svojstvo skupa hipotez, posmatra skup hipoteza i predstavlja jedinstven broj koji oznacava raznolikost tog skupa hipoteza

SAMPLING BIAS

- Šta je pošlo po zlu? Da li je ovo posledica probabilističkih garancija?

$$P[|E_{in} - E_{out}| > \epsilon] \leq \delta$$

- U ovom slučaju, nije δ krivo. Dobijeni rezultat je bio posledica sistematske greške (*bias*) u uzorkovanju:
  - 1948 telefoni su bili skupi – kuće sa telefonom su obično bile bogatije
  - Bogati ljudi su preferirali Dewey-a

- If the data is sampled in a biased way, learning will produce a similarly biased outcome

- Moramo se obezbediti da podaci budu reprezentativni u odnosu na ono šta želimo da pokažemo

- Praktičan primer: financial forecasting
  - Situacija je prilično nepredvidiva, pod uticajem neke glasine tržište može drastično da se promeni
  - Ako želimo da pronađemo šablon koji postoji u podacima, razmatramo „normalne“ periode u kojima se taj šablon vidi
  - Kada testiramo model, testiramo ga na pravom tržištu. Može se desiti da uvidimo da postoji sistematsko odstupanje

Poklapanje distribucija

- Jedan način da se borimo sa sistematskim greškama u uzorkovanju jeste poklapanje distribucija
- Pretpostavka uvedena kako bi *Hoeffding*-ova nejednakost važila:
  - Instance korišćene za obučavanje su odabrane iz iste distribucije kao i instance koje ćemo koristiti za testiranje
  - Ako imamo sistematsku grešku u uzorkovanju, ovo je narušeno



- Ako imamo pristup trening i test distribuciji, možemo proveriti da li se poklapaju, npr., u datom primeru su trening i test distribucije donekle različite
- Ako poznajemo ove distribucije, možemo:
  - dodeliti različite težine primerima trening skupa
  - Iz datih podataka ponovo uzorkovati trening skup tako da ispadne kao da je izvučen iz druge distribucije

Data snooping

- Princip: ako je skup podataka imao uticaja na bilo koji korak učenja, onda je mogućnost ovog skupa podataka da proceni ishod učenja kompromitovana

- Ovo je zamka u koju upadaju mnogi
  - Ima mnogo načina da se uhvatimo u nju
  - I veoma je privlačno da upadnemo u nju jer dobijamo bolje performanse
  - Manifestuje se na mnogo različitih načina

Dva rešenja za data snooping

1. Izbegavanje
   - Disciplina – stavite test podatke u sef i nemojte ga otvarati sve dok nemate finalnu hipotezu
2. Uračunajte i efekat data snooping-a u performanse
   - Kolika je kontaminacija podataka (VC dimenzija, ...)
   - Najteže je uraditi ako ručno pogledamo podatke – teško je modelovati sebe (koliki skup hipoteza je razmatran)